

The Agent: The Loop That Is Your 50 Percent Row

Chapter 4, objective 1.3c. Exhibit E7 decoded branch by branch, the seven stop_reason values, and the loop that never asks why it stopped.

The detective on a retainer. A workflow clerk follows a checklist. A detective is hired differently: find out why KF-2481 is stuck, here are your tools, here is your expense cap, report when done. Each clue decides the next move. Nobody scripted the investigation. The retainer has two clauses every stem cares about: the expense cap (budget) and report when done (exit). A detective without both is not employed. He is loose.

An **agent** is a model dynamically directing its own process and tool usage. Mechanically it is one thing only: **a loop around messages.create that keeps going while the model asks for tools and stops when it doesn't**. The field that steers the loop is stop_reason. The Foundations report says this mechanism cost you half your marks, so the book gives it the slowest treatment, and so does this part.

Exhibit E7, branch by branch

Lines	What E7 does	In plain words
1 to 3	MODEL set. BUDGET = 6. DRY_RUN read.	The retainer is signed. The expense cap is written in line two, before the loop exists. Law 3 is written before the loop, never as an afterthought at the bottom.
5 to 10	run_tool(call) handles the two known tools and returns a structured error for anything else.	The toolbag. Same promise shape as E1's phone book, now two tools. Line 10 matters: an unknown tool name returns an error result, not an exception. The detective learns his tool failed and can adapt. Crashing the loop would teach him nothing.
12 to 14	msgs starts with one user turn, the brief. Client is None on battery.	The case file opens with the brief. Everything the investigation learns is appended here, because the API is stateless. The file is the memory.
15	for turn in range(1, BUDGET + 1)	The loop is bounded by construction. The exam's favourite wrong option runs while True with the budget checked somewhere else. Somewhere else is nowhere.
16 to 17	On battery: print the turn and return a dry-run verdict.	Why exactly one line prints under DRY_RUN: the first turn returns before any paid call.
18 to 19	One messages.create per turn with tools and the whole case file.	One paid call per turn. The entire file goes up, one decision comes back. The response's stop_reason is the detective's note clipped to the front.
20	match r.stop_reason	The match statement is the whole objective. Every branch below is one value of that one field.
21 to 26	tool_use: append the assistant content, run every tool he asked for, append a user turn with all the tool_result blocks.	I need the phone. Three moves in order: file his request, run every tool, file the receipts matched by tool_use_id. Miss the first and the API rejects the receipts as orphans. The loop then comes round again, because the model's own reply decides whether there is a next turn. That sentence is your 50 percent row, verbatim.
27 to 28	end_turn: return the text.	Case closed, here is my report. The natural exit. Do not loop again out of politeness; a loop that calls the API once more after end_turn is paying for a conversation with someone who already left.
29 to 31	max_tokens: append the content, append a user turn saying Continue.	The report hit the page limit mid-sentence. Truncated is not finished. File what arrived and ask for the rest. Treating max_tokens as success ships half a verdict, the silent failure's favourite door.
32 to 33	pause_turn: append the content and continue.	A server-side tool loop pausing for breath. Return the content as-is to resume. A newer value; most workbooks do not know it exists.
34 to 35	refusal: escalate with the reason model refused.	The detective declines the case. The wrong reflex is to retry automatically. The right one is a different path, here a human.
36 to 37	The wildcard: escalate with the unhandled value.	stop_sequence, model_context_window_exceeded, and whatever ships next quarter all land in escalation instead of an unhandled crash. A total match is what the phrase orchestration-layer safeguard looks like at line level.
38	After the loop: escalate(brief, budget exhausted).	The floor. Budget spent, case not closed. Every session ends in one of two ways: a completed resolution or a human hand-off with a reason. Never a silent stop.
40 to 43	escalate() returns a line naming Meera's desk, the reason and the brief.	Escalation is a first-class outcome, not an error string. Meera's desk learns why the machine gave up. That is a Domain 6 hand-off package in miniature.

Everyday example for stop_reason

Think of how a phone call ends. *Call me back after you check that* is tool_use. *That is all, goodbye* is end_turn. The line cutting out mid-sentence is max_tokens. *Hold please* is pause_turn. *I will not discuss that* is refusal. You would never treat a cut-off line as the end of the conversation. The loop must not either.

Drift: how many stop_reason values exist. Most study material teaches four stop_reason values: end_turn, max_tokens, stop_sequence, tool_use. The brief's reading of current docs lists seven, adding pause_turn (a server-tool loop paused, default limit 10 iterations, send content back to continue), refusal (declined; read stop_details), and model_context_window_exceeded (treat as truncated). All arrive as HTTP 200. A stop_reason is a status, never an error. The API reference loaded earlier in this session confirms the first six and that refusal carries stop_details; carry the seventh as the brief states it.

Trap: the loop that never asks why it stopped. Distractors love code that checks for tool_use and treats everything else as the finished answer. max_tokens then ships truncations; refusal ships an apology as a verdict. The tell in such stems is an **else that swallows**. This is the silent-failure family grown up, and at Professional level it is also a diagnosis question in Domain 4.

LAW 3 · Every loop needs a budget and an exit. E6 had max_rounds and PASS. E7 has BUDGET and end_turn, plus escalation as the floor beneath both. Two organs, every loop, no exceptions. When a stem's code lacks one, that absence is the answer.

Workflow, agent, or augmented LLM: the 1.3 decision in one table

The stem says	Choose	Because
Steps identical every time; accuracy over latency	Prompt chaining	fixed path plus gates
Known categories, different handling each	Routing	cheap sort, specialist wards
Independent subtasks plus a latency budget	Parallel (sectioning)	slowest-scout wall clock
High-stakes judgement, tolerance for 3x cost	Parallel (voting)	confidence by majority
Subtasks unpredictable, but delegate-then-synthesise is fixed	Orchestrator-workers	model plans, code steers
Clear rubric, drafts improve measurably	Evaluator-optimizer	criteria exist
Path itself unknowable; each result steers the next act	Agent	model directs, with budget and exit
One question, tools at hand, no multi-step anything	Augmented LLM	the cell suffices

Domain 3 connection: the lab's E6 router is a 1.3 routing workflow. The One Ticket walk, where every hop was fixed in advance, is a workflow end to end. The only agent-shaped moment in the whole Domain 3 series was the hypothetical tangled case sent to the agent tier with a budget.

***The one thing to remember.** An agent is a bounded loop around one call, steered by stop_reason, with a budget, an exit and an escalation floor. The model's own reply decides whether there is a next turn. Handle every value of that field, or the one you ignore becomes the failure you ship.*